🎓 OXFORD CRM MASTER TECHNICAL HANDBOOK

The Oxford Computers — WhatsApp CRM & Automation Platform

Version: Phase 9.6 (Production)

Prepared by: Antigravity AI

Date: June 2026

Scope: Complete system reference — 60+ pages

> [!IMPORTANT]

> This document is the single source of truth for the Oxford CRM system. It is designed so that any new AI coding session can reconstruct full project context by reading this handbook alone.

TABLE OF CONTENTS

---

1. PROJECT HISTORY & EVOLUTION

1.1 Background: Artisun → Oxford CRM

The Oxford Computers is a computer education institute located in **Malayinkeezhu Junction, Thiruvananthapuram, Kerala**. They offer Kerala State Rutronix approved diploma and certificate courses in IT, digital marketing, accounting, and computing.

The CRM system began as a simple WhatsApp chatbot and evolved into a full-featured CRM platform across 9+ development phases spanning 2025–2026.

Era	Description
Pre-CRM	Manual WhatsApp responses, no tracking
Phase 1–2	Basic webhook bot with in-memory state
Phase 3	PostgreSQL persistence, follow-up scheduler
Phase 4A–4D	CRM expansion fields, lead scoring, message logging
Phase 5A–5C	CRM Timeline, conversation search & filters
Phase 6A–6G	Lead events, intelligence engine, campaigns
Phase 7A–7F	Full analytics suite (funnel, staff, source, admission)
Phase 8.1–8.8	Revenue analytics, lead portfolio, action center, health dashboard

| Phase 9.1–9.6 | Staff registry, task management, operations intelligence, automation |

1.2 Key Architectural Decisions

1. **No foreign key constraints** — All tables use `phone` as a soft join key. This allows partial data without referential integrity failures.

2. **Append-only event log** — `LeadEvent` rows are never updated or deleted. History is immutable.

3. **Dual logging system** — `MessageLog` (raw technical) + `ConversationMessage` (CRM structured) are intentionally separate.

4. **DB-backed state** — `StateProxy` dict auto-persists to DB on every key assignment, replacing the old in-memory dict.

5. **Thread-safe logging** — All log operations from daemon threads use `_in_thread` wrappers that open their own Flask app context.

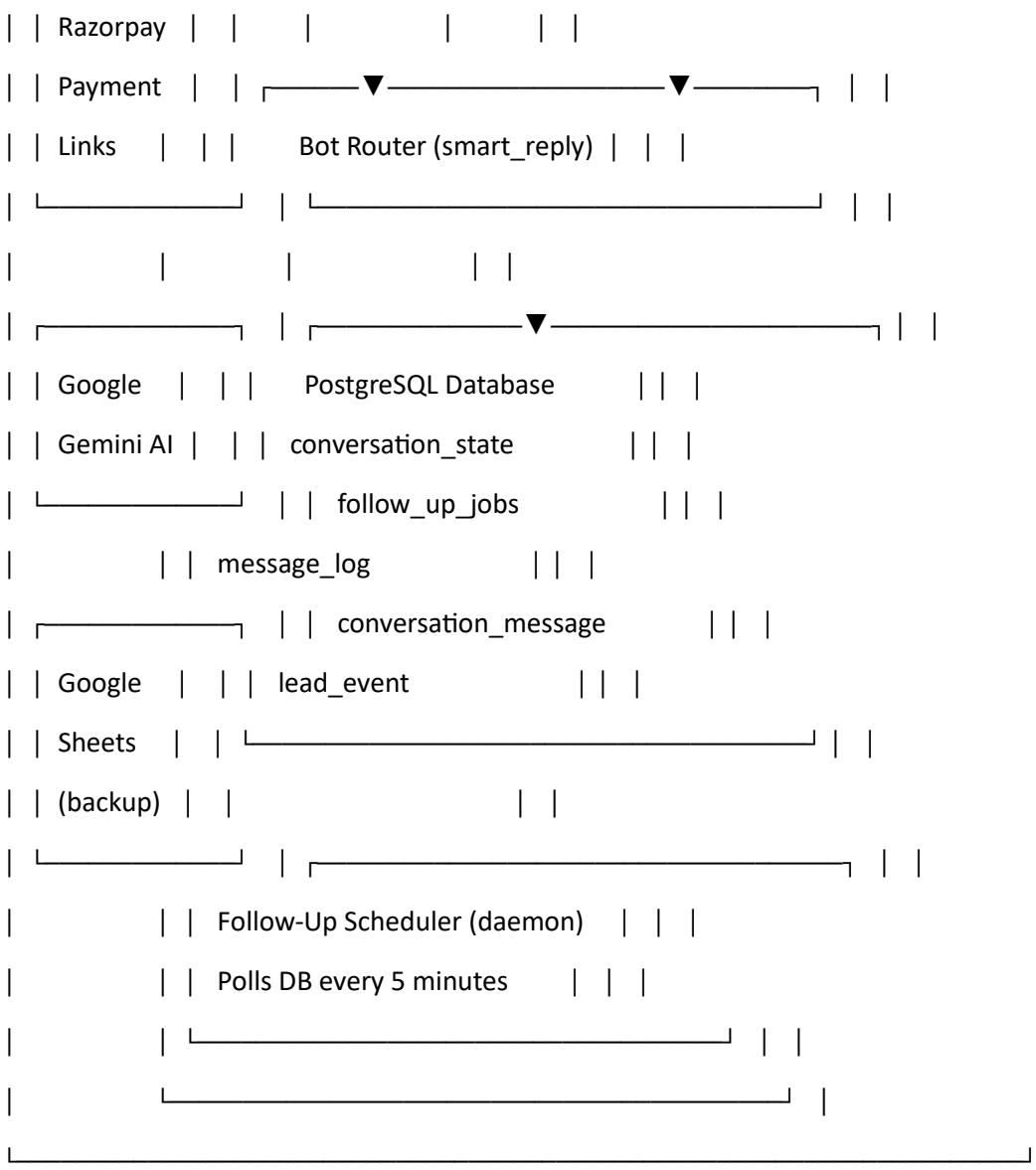
6. **Zero N+1 queries** — All analytics functions use bulk queries + Python-memory aggregation.

7. **Staff as JSON file** — `staff_master.json` file, NOT a DB table. This avoids a migration for every staff change.

---

2. SYSTEM ARCHITECTURE OVERVIEW

```
┌──────────────────────────────────────────────────────────────────────┐
│                    RAILWAY PRODUCTION                                │
│                                                                      │
│  ┌──────────────┐   ┌──────────────────────────────────────────┐     │
│  │              │   │                                          │     │
│  │  WhatsApp    │──▶│          Flask Application               │     │
│  │  Cloud API   │   │                                          │     │
│  │  (Meta)      │   │  ┌──────────────┐  ┌──────────────┐      │     │
│  │              │   │  │              │  │              │      │     │
│  └──────────────┘   │  │   Webhook    │  │  Admin/CRM   │      │     │
│                     │  │  Blueprint   │  │  Blueprint   │      │     │
│  ┌──────────────┐   │  └──────┬───────┘  └──────┬───────┘      │     │
```




```

### 2.1 Module Map

```
oxford-whatsapp_2/
 ├── run.py   # App entry point
 ├── Procfile   # Railway: web: gunicorn run:app
 ├── requirements.txt   # Python dependencies
 ├── .env.example   # Environment variable template
 ├── app/
```
```

```
|   ├── __init__.py          # create_app() factory
|   ├── config.py            # All env-var reads
|   ├── extensions.py        # db, migrate (SQLAlchemy, Flask-Migrate)
|   ├── models.py            # 5 DB models
|   ├── state.py             # StateProxy, get_or_create_state()
|   ├── bot/
|   |   ├── constants.py     # ALL_COURSES, COURSE_FEES, OFFER_MENU, buttons
|   |   ├── router.py        # smart_reply() — main conversation engine
|   |   ├── objections.py    # Objection detection & handling
|   |   └── prompts.py       # Gemini AI prompt templates
|   ├── routes/
|   |   ├── webhook.py       # /webhook GET (verify) + POST (receive)
|   |   ├── admin.py         # All /crm/* routes (3957 lines)
|   |   ├── broadcast.py     # /broadcast endpoint
|   |   └── health.py        # /health endpoint
|   ├── services/
|   |   ├── ai_service.py    # gemini_reply(), smart_fallback()
|   |   ├── campaign_service.py # start_campaign()
|   |   ├── crm_service.py   # update_lead_status(), save_lead_to_sheets()
|   |   ├── followup_service.py # scheduler + FOLLOWUP_TEMPLATES
|   |   ├── log_service.py   # log_message(), save_conversation_message(), log_lead_event()
|   |   └── whatsapp_service.py # send_text(), send_reply()
|   └── data/
|       └── staff_master.json # Staff registry (JSON, not DB table)
└── templates/
    ├── panel.html           # Legacy admin panel
    ├── crm_leads.html       # Main lead list dashboard
    ├── crm_lead_detail.html  # Per-lead detail + timeline (102KB)
    ├── crm_analytics.html   # Funnel analytics
    ├── crm_staff_performance.html
```

```
    ├── crm_source_analytics.html
    ├── crm_admission_analytics.html
    ├── crm_revenue_analytics.html
    ├── crm_health.html
    ├── crm_action_center.html
    ├── crm_operations.html
    ├── crm_staff_management.html
    ├── crm_staff_dashboard.html
    ├── crm_my_leads.html
    ├── crm_my_tasks.html
    ├── crm_admin_tasks.html
    ├── crm_unassigned_leads.html
    ├── crm_reassignment_center.html
    ├── crm_staff_performance_detail.html
    ├── crm_staff_workload.html
    └── campaigns.html
```

---

3. TECHNOLOGY STACK & DEPENDENCIES

3.1 Core Stack

Layer	Technology	Version
Language	Python	3.11+
Web Framework	Flask	Latest
ORM	Flask-SQLAlchemy	Latest
Migrations	Flask-Migrate (Alembic)	Latest

| Database | PostgreSQL | 15+ (Railway) |

| DB Driver | psycopg2-binary | Latest |

| WSGI Server | Gunicorn | Latest |

| AI | Google Gemini | google-genai |

| Spreadsheet | Google Sheets | gspread + oauth2client |

| HTTP | requests | Latest |

3.2 Environment Variables

| Variable | Purpose | Default |

|----------|---------|---------|

| `DATABASE_URL` | PostgreSQL connection string | **REQUIRED** |

| `ACCESS_TOKEN` | WhatsApp Cloud API access token | "" |

| `PHONE_NUMBER_ID` | WhatsApp phone number ID | "" |

| `VERIFY_TOKEN` | Webhook verification token | "oxford2026" |

| `ADMIN_KEY` | CRM admin access key | "oxford_admin_2026" |

| `GEMINI_API_KEY` | Google Gemini API key | "" |

| `SECRET_KEY` | Flask session secret | "oxford-crm-local-dev-key" |

| `BROADCAST_API_KEY` | Broadcast endpoint key | "oxford_broadcast_2026" |

| `SHEETS_ID` | Google Sheets document ID | "" |

| `GOOGLE_CREDENTIALS` | Google service account JSON | "{}" |

> [!CAUTION]

> `DATABASE_URL` is **mandatory**. The app raises `RuntimeError` on startup if missing. Railway injects this automatically.

3.3 WhatsApp API Config

- **API URL:** `https://graph.facebook.com/v19.0/{PHONE_NUMBER_ID}/messages`

- **API Version:** v19.0

- **Message Types Supported:** `text`, `interactive` (button_reply, list_reply), `button`

- **Unsupported types:** Silently ignored (returns 200)

---

4. DATABASE SCHEMA DOCUMENTATION

4.1 Schema Overview

```
PostgreSQL Database
 ├── conversation_state       ← Primary lead record (one per phone)
 ├── follow_up_jobs           ← Scheduled follow-up message queue
 ├── message_log              ← Raw technical event log (append-only)
 ├── conversation_message     ← CRM-structured timeline (append-only)
 └── lead_event               ← Named business events (append-only)
```

4.2 `conversation_state` Table

```sql
CREATE TABLE conversation_state (
    id              INTEGER PRIMARY KEY,
    phone           VARCHAR(20) UNIQUE NOT NULL,   -- Indexed
    name            VARCHAR(200) DEFAULT '',
    stage           VARCHAR(50)  DEFAULT 'new',
    course          VARCHAR(200) DEFAULT '',
    goal            VARCHAR(50)  DEFAULT '',
    batch_time      VARCHAR(100) DEFAULT '',
    offer_course    VARCHAR(50)  DEFAULT '',
    last_msg        VARCHAR(50)  DEFAULT '',
    last_text       TEXT         DEFAULT '',
```

```
    updated_at      TIMESTAMP    NOT NULL,
    -- Phase 4A additions:
    created_at      TIMESTAMP    NOT NULL,
    lead_status     VARCHAR(50)  DEFAULT 'Lead',
    assigned_staff  VARCHAR(100),
    lead_score      INTEGER      DEFAULT 0,
    is_admitted     BOOLEAN      DEFAULT FALSE,
    notes           TEXT
);
```

4.3 `follow_up_jobs` Table

```sql
CREATE TABLE follow_up_jobs (
    id          INTEGER PRIMARY KEY,
    phone       VARCHAR(20) NOT NULL,   -- Indexed
    name        VARCHAR(200) DEFAULT '',
    send_at     TIMESTAMP   NOT NULL,
    message     TEXT        NOT NULL,
    day         INTEGER     NOT NULL,   -- 1, 3, or 7
    done        BOOLEAN     DEFAULT FALSE,  -- Indexed
    created_at  TIMESTAMP   DEFAULT NOW()
);
```

4.4 `message_log` Table

```sql
CREATE TABLE message_log (
    id          INTEGER PRIMARY KEY,
```

```
    phone       VARCHAR(20) NOT NULL,   -- Indexed
    direction   VARCHAR(10) NOT NULL,   -- "inbound" | "outbound"
    message_type VARCHAR(20) NOT NULL,  -- "user" | "ai" | "followup" | "manual"
    message_text TEXT,
    meta_json   TEXT,                   -- Optional JSON metadata
    created_at  TIMESTAMP   NOT NULL,
    INDEX idx_msg_phone_created (phone, created_at)
);
```

4.5 `conversation_message` Table (CRM Timeline)

```sql
CREATE TABLE conversation_message (
    id          INTEGER PRIMARY KEY,
    phone       VARCHAR(20)  NOT NULL,
    direction   VARCHAR(10)  NOT NULL,   -- "incoming" | "outgoing"
    message     TEXT,
    message_type  VARCHAR(20),           -- "text" | "interactive" | "button" | "template" | "system"
    source      VARCHAR(20),             -- "user" | "ai" | "manual" | "followup" | "system" | "campaign"
    staff_name  VARCHAR(100),            -- Populated for manual sends only
    wa_message_id VARCHAR(100),          -- WhatsApp message ID (deduplication)
    created_at  TIMESTAMP    NOT NULL,
    INDEX idx_conv_msg_phone_created (phone, created_at),
    INDEX idx_conv_msg_wa_id (wa_message_id)
);
```

4.6 `lead_event` Table (Business Events)

```sql
```

```
CREATE TABLE lead_event (
    id          INTEGER PRIMARY KEY,
    phone       VARCHAR(20) NOT NULL,   -- Indexed
    event_type VARCHAR(50) NOT NULL,
    event_data TEXT,                    -- Optional JSON or plain string
    created_at TIMESTAMP   NOT NULL,
    INDEX idx_lead_event_phone_created (phone, created_at)
);
```

4.7 Index Strategy

Index	Table	Columns	Purpose
`idx_msg_phone_created`	message_log	phone, created_at	Timeline queries
`idx_conv_msg_phone_created`	conversation_message	phone, created_at	CRM history
`idx_conv_msg_wa_id`	conversation_message	wa_message_id	Deduplication
`idx_lead_event_phone_created`	lead_event	phone, created_at	Event history
(unique)	conversation_state	phone	One row per lead
(index)	follow_up_jobs	done	Pending job queries

---

5. CONVERSATIONSTATE — FIELD-BY-FIELD REFERENCE

This is the most critical table. One row per WhatsApp phone number. It is the "lead record."

Field	Type	Description	Allowed Values
`id`	INTEGER	Auto-increment primary key	—
`phone`	VARCHAR(20)	WhatsApp number with country code	e.g. `919447329972`

| `name` | VARCHAR(200) | Lead's WhatsApp display name | Any string |

| `stage` | VARCHAR(50) | Current conversation stage | See stage table below |

| `course` | VARCHAR(200) | Course the lead is interested in | From ALL_COURSES canonical names |

| `goal` | VARCHAR(50) | Career goal selected | `job`, `business`, `basic`, `accounting` |

| `batch_time` | VARCHAR(100) | Preferred demo batch time | `Morning (9–11 AM)`, `Afternoon (12–2 PM)`, `Evening (5–7 PM)` |

| `offer_course` | VARCHAR(50) | Short code from OFFER_MENU | `CWPDE`, `DCA`, `AIDM`, `PGDCA` |

| `last_msg` | VARCHAR(50) | ISO timestamp of last message | `2026-06-01T10:30:00` |

| `last_text` | TEXT | Raw text of last message | Any string |

| `updated_at` | TIMESTAMP | Auto-updated on any change | — |

| `created_at` | TIMESTAMP | First contact timestamp | — |

| `lead_status` | VARCHAR(50) | CRM pipeline status | See status table below |

| `assigned_staff` | VARCHAR(100) | Counselor name (normalized) | From `staff_master.json` |

| `lead_score` | INTEGER | Manual score (0–100) | 0–100 |

| `is_admitted` | BOOLEAN | Admission flag | True / False |

| `notes` | TEXT | Staff free-text notes | Any string |

5.1 Stage Values

Stage	Meaning	Next Stage
`new`	Bot not yet responded	`goal_selection`
`goal_selection`	Waiting for goal choice (1–5)	`course_recommendation` or `not_sure`
`course_recommendation`	Goal selected, showing courses	`course_viewed`
`course_viewed`	Lead viewed a course detail	`demo_time_ask` or `offer_menu`
`demo_time_ask`	Asking preferred batch time	`demo_date_ask`
`demo_date_ask`	Asking preferred date	`demo_booked`
`demo_booked`	Demo confirmed	(terminal or re-engage)
`offer_menu`	Showing payment offers	`payment_pending`
`payment_pending`	Payment link sent	`enrolled`
`enrolled`	Payment confirmed	(terminal)

| `not_sure` | Help me choose path | `goal_selection` |

| `done` | Lead exited | (terminal or re-engage) |

5.2 Lead Status Values

| Status | Meaning |

|--------|---------|

| `Lead` | Default — new, uncontacted |

| `Contacted` | Staff has been in touch |

| `Interested` | Lead showing interest |

| `Viewed: {course}` | Bot auto-set when course viewed |

| `Demo Booked: {details}` | Bot auto-set when demo confirmed |

| `Follow-up Day {N} Sent` | Auto-set by scheduler |

| `Call Requested` | Lead asked to be called |

| `Office Visit Interested` | Lead wants to visit |

| `Payment Received: {txn}` | Bot auto-set on payment |

| `Enrolled` | Auto-promoted on admission |

---

6. LEADEVENT CATALOG WITH JSON PAYLOAD EXAMPLES

The `lead_event` table is an **append-only** audit trail of named business events. Events are NEVER updated or deleted.

6.1 Event Type Reference

| Event Type | Triggered By | event_data Format | Score Points |

|------------|-------------|-------------------|-------------|

| `LEAD_CREATED` | webhook.py — new phone | NULL | +2 |

| `FIRST_MESSAGE_RECEIVED` | webhook.py — new phone | NULL | +3 |

| `AI_RESPONSE_SENT` | webhook.py — new phone | NULL | +5 |

| `COURSE_VIEWED` | router.py — course selected | plain string: course name | +10 |

| `PLACEMENT_ASKED` | router.py — placement inquiry | NULL | +15 |

| `FEES_REQUESTED` | router.py — fees asked | plain string: course name or NULL | +20 |

| `DEMO_REQUESTED` | router.py — demo booked | NULL | +25 |

| `PAYMENT_PENDING` | router.py — offer menu selection | NULL | +30 |

| `COURSE_ENQUIRY` | admin.py — crm_lead_update | JSON: `{"course": "name"}` | — |

| `COURSE_ADMISSION` | admin.py — crm_lead_update / crm_course_admissions | JSON: `{"course": "name", "staff": "name"}` | — |

| `LEAD_REASSIGNED` | admin.py — crm_lead_update | JSON: `{"from": "old", "to": "new", "by": "Admin"}` | — |

| `MANUAL_MESSAGE` | admin.py — crm_lead_send | JSON: `{"staff": "name"}` | — |

| `FOLLOW_UP_TASK` | admin.py — task creation | JSON: see below | — |

| `FOLLOW_UP_COMPLETED` | admin.py — task completion | JSON: `{"task_id": "uuid"}` | — |

6.2 JSON Payload Examples

COURSE_VIEWED (plain string):

```
event_data = "PGDCA"
```

FEES_REQUESTED (plain string or NULL):

```
event_data = "AIDM Digital Marketing"
event_data = NULL  (when no course selected yet)
```

COURSE_ENQUIRY (JSON):

```json
{"course": "Python Programming"}
```

COURSE_ADMISSION (JSON):

```json
{"course": "PGDCA", "staff": "Kiran"}
```

LEAD_REASSIGNED (JSON):

```json
{"from": "Anjali", "to": "Kiran", "by": "Admin"}
```

MANUAL_MESSAGE (JSON):

```json
{"staff": "Kiran"}
```

FOLLOW_UP_TASK (JSON):

```json
{
  "task_id": "uuid-string",
  "task": "Call lead about PGDCA fees",
  "due_date": "2026-06-10",
  "staff": "Kiran",
  "phone": "919447329972"
}
```

FOLLOW_UP_COMPLETED (JSON):

```json
{"task_id": "uuid-string"}
```

6.3 Lead Scoring Formula

```python
EVENT_SCORE_MAP = {
    "LEAD_CREATED":          2,
    "FIRST_MESSAGE_RECEIVED": 3,
    "AI_RESPONSE_SENT":      5,
    "COURSE_VIEWED":         10,
    "PLACEMENT_ASKED":       15,
    "FEES_REQUESTED":        20,
    "DEMO_REQUESTED":        25,
    "PAYMENT_PENDING":       30,
}

auto_score  = sum(EVENT_SCORE_MAP.get(event_type, 0) for event_type in unique_event_types)
final_score = min((manual_score or 0) + auto_score, 100)

# Temperature thresholds:
# final_score >= 80  → HOT
# final_score >= 50  → WARM
# final_score <  50  → COLD
```

Maximum possible auto-score: 2+3+5+10+15+20+25+30 = **110** (capped at 100)

6.4 Recommended Action Logic

```python
if "PAYMENT_PENDING" in events:    action = "Payment Follow-up"
elif "DEMO_REQUESTED" in events:   action = "Send Demo"
```

```
elif "FEES_REQUESTED" in events:   action = "Send Fees"
elif "PLACEMENT_ASKED" in events:  action = "Discuss Placement"
elif final_score >= 80:            action = "Call Today"
elif "LEAD_CREATED" and score<=15: action = "Qualify Lead"
else:                              action = "Admission Follow-up"
```

---

7. BOT CONVERSATION ENGINE

7.1 Message Flow

```
WhatsApp User Message
      ↓
webhook.py (POST /webhook)
      ↓
  Parse message type (text / interactive / button)
      ↓
  Log inbound → message_log (daemon thread)
  Log inbound → conversation_message (daemon thread)
  Fire LEAD_CREATED + FIRST_MESSAGE_RECEIVED (if new lead)
      ↓
  smart_reply(msg_text, name, phone, is_new_lead)
      ↓
  ┌──────────────────────────────────────────────────┐
  │            router.py smart_reply()               │
  │                                                  │
  │  1. Load/create StateProxy (DB-backed state dict)  │
  │  2. Special keywords (EXIT, greetings, DEMO, etc.) │
```

```
  | 3. Objection detection                    |
  | 4. Stage-specific routing                 |
  | 5. Keyword-to-course matching             |
  | 6. AI fallback (Gemini)                   |
  | 7. smart_fallback() as last resort        |
  └──────────────────────────────────────────┘
      ↓
  send_reply(phone, reply_text, button_preset)
      ↓
  Log outbound → message_log (daemon thread)
  Log outbound → conversation_message (daemon thread)
  Fire AI_RESPONSE_SENT (if new lead, daemon thread)
      ↓
  schedule_followups() (if new lead)
```

7.2 StateProxy — How Auto-Save Works

```python
class StateProxy(dict):
    """
    A dict subclass. Any assignment auto-persists to DB.
    """
    def __setitem__(self, key, value):
        super().__setitem__(key, value)
        _db_save(self._phone, self)  # ← DB write on EVERY assignment

# Usage (from router.py):
st = get_or_create_state(phone, name)
st["stage"] = "goal_selection"  # ← This instantly writes to DB
st["course"] = "PGDCA"          # ← This instantly writes to DB
```

```

**Fields persisted by StateProxy:**

`name`, `stage`, `course`, `goal`, `batch_time`, `offer_course`, `last_msg`, `last_text`

**Fields NOT auto-saved (manual DB writes only):**

`lead_status`, `assigned_staff`, `lead_score`, `is_admitted`, `notes`

### 7.3 Conversation Stages Flowchart

```
[new / done / enrolled / goal_selection]
        ↓ (greeting or new lead)
   [goal_selection]
   Ask: 1=Job 2=Business 3=Basic 4=Accounting 5=Not sure
        ↓
   [course_recommendation]
   Show goal-based course list (1–3 courses)
        ↓ (select number)
   [course_viewed]
   Show course detail + TRUST + DEMO CTA
   Fire: COURSE_VIEWED event
        ↓
   ┌──────────────────────────────────────┐
   │ User says: DEMO / FEES / COURSES   │
   └──────────────────────────────────────┘
        ↓ DEMO
   [demo_time_ask] → Morning/Afternoon/Evening
        ↓
   [demo_date_ask] → Any text date
        ↓
```
```

[demo_booked] → Confirmation sent

Fire: update_lead_status("Demo Booked: ...")

↓ FEES

Show FULL_FEE_TABLE or course-specific fee

Fire: FEES_REQUESTED event

↓ PAY / OFFER

[offer_menu] → Show 4 payment options

↓ (select 1–4)

[payment_pending] → Send Razorpay link

↓ (user sends TXN ID)

[enrolled] → Payment confirmed

Fire: update_lead_status("Payment Received: ...")

```

### 7.4 All Courses Catalog

| Index | Course Name | Duration | Fee | Razorpay Link |
|---|---|---|---|---|
| 1 | PGDCA | 12 Months | ₹15,999 | rzp.io/rzp/KAQ2C7t |
| 2 | AIDM Digital Marketing | 6 Months | ₹19,999 | rzp.io/rzp/vF76sj7Y |
| 3 | SAP Financial Accounting | 4-6 Months | ₹15,999 | — (counselor) |
| 4 | Python Programming | 3 Months | ₹4,499 | — |
| 5 | GST & Payroll | 6 Months | ₹18,999 | — |
| 6 | DCA Fast Track | 6 Months | ₹6,400 | rzp.io/rzp/mJPPtM9x |
| 7 | Computer Teacher Training | 1 Year | ₹11,999 | — |
| 8 | Corporate Business Accounting | 1 Year | ₹40,000 | — |
| 9 | Word Processing & Data Entry | 6 Months | ₹4,800 | rzp.io/rzp/xkWdKtd |
| 10 | Professional Web Designing | 6 Months | ₹8,800 | — |

### 7.5 Goal-to-Course Mapping
```

| Goal | Recommended Courses |

|------|---------------------|

| `job` | PGDCA, Python, Web Designing, DCA Fast Track |

| `business` | AIDM Digital Marketing, Web Designing, Python |

| `basic` | DCA Fast Track, Word Processing, Computer Teacher Training |

| `accounting` | SAP Financial Accounting, GST & Payroll, Corporate Business Accounting |

7.6 Course Name Normalization (Phase 7F.2)

Aliases are resolved at READ time only — no DB writes:

```python
COURSE_NAME_ALIASES = {
    "aidm":             "AIDM Digital Marketing",
    "dca":              "DCA Fast Track",
    "cwpde":            "Word Processing & Data Entry",
    "pgdca":            "PGDCA",
    "sap accounting":   "SAP Financial Accounting",
    "computer teaching": "Computer Teacher Training",
    "web designing":    "Professional Web Designing",
    ...
}
```

---

8. COMPLETE CRM ROUTE CATALOG

All CRM routes require `?key=ADMIN_KEY` query parameter.

8.1 Webhook Routes (`webhook_bp`)

| Method | Route | Purpose |

|--------|-------|---------|

| GET | `/webhook` | WhatsApp webhook verification (hub.challenge) |

| POST | `/webhook` | Receive WhatsApp messages |

8.2 Admin/CRM Routes (`admin_bp`)

| Method | Route | Purpose | Template |

|--------|-------|---------|----------|

| GET | `/panel` | Legacy admin panel (raw HTML file) | panel.html |

| GET | `/stats` | JSON stats endpoint (X-Admin-Key header) | JSON |

| POST | `/trigger-followup` | Manual follow-up trigger | JSON |

| GET | `/crm/leads` | Main lead list with intelligence | crm_leads.html |

| GET | `/crm/lead/<phone>` | Lead detail + timeline + events | crm_lead_detail.html |

| POST | `/crm/lead/<phone>/update` | Update lead CRM fields | Redirect |

| POST | `/crm/lead/<phone>/send` | Send manual WhatsApp message | Redirect |

| GET | `/crm/analytics` | Funnel analytics | crm_analytics.html |

| GET | `/crm/staff-performance` | Staff leaderboard | crm_staff_performance.html |

| GET | `/crm/source-analytics` | Lead source attribution | crm_source_analytics.html |

| GET | `/crm/admission-analytics` | Admission breakdown | crm_admission_analytics.html |

| GET | `/crm/revenue-analytics` | Revenue dashboard (Phase 8.1) | crm_revenue_analytics.html |

| GET | `/crm/health` | CRM health & data quality | crm_health.html |

| GET | `/crm/action-center` | Prioritized action buckets | crm_action_center.html |

| GET | `/crm/operations` | Operations command center | crm_operations.html |

| GET/POST | `/crm/staff-management` | Staff registry CRUD | crm_staff_management.html |

| GET | `/crm/campaigns` | Campaign dashboard | campaigns.html |

| POST | `/crm/campaigns/preview` | Preview audience count | JSON |

| POST | `/crm/campaigns/send` | Send campaign | Redirect |

| POST | `/crm/course-admissions/<phone>` | Multi-course admission | Redirect |

| GET | `/crm/my-leads` | Staff view: own leads | crm_my_leads.html |

| GET | `/crm/my-tasks` | Staff view: own tasks | crm_my_tasks.html |

| GET | `/crm/admin-tasks` | Admin: all tasks overview | crm_admin_tasks.html |

| GET | `/crm/unassigned-leads` | Unassigned leads list | crm_unassigned_leads.html |

| GET | `/crm/reassignment-center` | Bulk reassignment UI | crm_reassignment_center.html |

| GET | `/crm/staff-dashboard` | Staff performance dashboard | crm_staff_dashboard.html |

| GET | `/crm/staff-performance-detail/<name>` | Per-staff deep dive | crm_staff_performance_detail.html |

| GET | `/crm/staff-workload` | Staff workload overview | crm_staff_workload.html |

8.3 Other Routes

| Method | Route | Purpose |

|--------|-------|---------|

| GET/POST | `/broadcast` | Bulk message endpoint |

| GET | `/health` | Uptime health check |

8.4 Route Authentication Patterns

Header-based (JSON APIs):

```
X-Admin-Key: {ADMIN_KEY}
```

Query-param-based (HTML pages):

```
?key={ADMIN_KEY}
```

---

9. DASHBOARD ARCHITECTURE

9.1 Main Lead List (`/crm/leads`)

Data loaded on every page request:

```python
# 1. Paginated lead query (25 per page)
# 2. All lead_scores + updated_at (for intelligence calc)
# 3. All LeadEvent phone+event_type (for auto-scoring)
# 4. Latest ConversationMessage per phone (for needs_reply)
# 5. Pending FollowUpJob phones (for badge)

# Computed for each lead:
intelligence = calculate_lead_intelligence(manual_score, events)
health = calculate_lead_health(...)
lead.needs_reply = phone in needs_reply_phones
lead.has_pending_fu = phone in pending_fu_phones
```

KPI Cards displayed:

- Total Leads count
- HOT Leads count (score ≥ 80)
- Call Today count (recommended action == "Call Today")
- Needs Reply count (last message was incoming)
- Critical Leads count (aging_status == "Critical")
- Pending Follow-ups count

Filters available:

- Search (phone / name ILIKE)
- Stage filter (dropdown)
- Admitted filter (yes / no)

9.2 Lead Detail (`/crm/lead/<phone>`)

Data loaded:

```python
lead = ConversationState (single lead)
logs = MessageLog (last 100, desc)
timeline = ConversationMessage (filtered, last 100)
events = LeadEvent (all, asc)
unified_timeline = merge of events + messages, sorted by created_at
intelligence = calculate_lead_intelligence(lead.lead_score, events)
health = calculate_lead_health(...)
course_journey = {enquiries: [...], admissions: [...]}
event_course_map = {event_id: course_name}  # pre-parsed JSON
event_payload_map = {event_id: dict}  # for task/reassign events
portfolio = calculate_lead_portfolio(lead, events, course_journey)
active_staff = list from staff_master.json
task_summary = {open, overdue, completed}
task_map = {task_id: payload_dict}
```

9.3 Lead Intelligence Engine

```python
def calculate_lead_intelligence(manual_score, events):
    unique_event_types = set(e.event_type for e in events)
    auto_score = sum(EVENT_SCORE_MAP.get(et, 0) for et in unique_event_types)
    final_score = min((manual_score or 0) + auto_score, 100)

    temperature = "HOT" if final_score >= 80 else "WARM" if final_score >= 50 else "COLD"
    # ... recommended_action logic (see Section 6.4)
    return {"final_score", "temperature", "recommended_action", "_events"}
```

```

### 9.4 Lead Health Engine

```python
def calculate_lead_health(updated_at, created_at, latest_msg_time,
                          latest_event_time, intelligence, needs_reply):
    latest_activity = max(all non-null timestamps)
    days_inactive = (now - latest_activity).days

    aging_status:
      0–2 days  → "Fresh"
      3–6 days  → "Attention"
      7–13 days → "Aging"
      14+ days  → "Critical"

    escalation triggers:
      HOT + Critical      → "🚨 HOT Lead Ignored"
      needs_reply + 1 day → "💬 Waiting For Reply"
      FEES + 7 days       → "💰 Fees Follow-up Needed"
      DEMO + 7 days       → "🎓 Demo Follow-up Needed"
```

---

10. STAFF MANAGEMENT ARCHITECTURE

10.1 Staff Registry (JSON File, NOT Database)

Location: `app/data/staff_master.json`
```

Format:

```json
{
  "KIRAN": {
    "display_name": "Kiran",
    "role": "STAFF",
    "active": true
  },
  "ANJALI": {
    "display_name": "Anjali",
    "role": "MANAGER",
    "active": true
  }
}
```

Why JSON not DB? Avoids Flask-Migrate schema changes for every staff addition.

10.2 Staff Operations

Operation	Route	Logic
Add staff	POST /crm/staff-management (action=add)	Write to JSON file
Edit staff	POST /crm/staff-management (action=edit)	Update JSON file
Toggle active	POST /crm/staff-management (action=toggle)	Flip `active` field
Deactivate safety	Before deactivate	Check assigned lead count; block if > 0

10.3 Staff Name Normalization

```python
def normalize_staff_name(name):
```

```
    """'kiran', 'KIRAN', ' Kiran ' → 'Kiran'"""
    if not name: return "Unassigned"
    cleaned = name.strip()
    if not cleaned: return "Unassigned"
    return cleaned.title()
```

Critical rule: All staff name comparisons use normalized form. CRM Health dashboard detects and penalizes duplicate naming variants (e.g., "Kiran" vs "kiran" in DB).

10.4 Staff Lead Assignment

- Staff is assigned via `/crm/lead/<phone>/update` form
- `assigned_staff` field in `conversation_state` stores the staff name
- On reassignment, `LEAD_REASSIGNED` event is logged with from/to/by
- If admission marked and no staff assigned → **hard block** (admission rejected)

10.5 Staff-Specific Views

View	Route	Shows
My Leads	/crm/my-leads?staff=Name	Leads assigned to this staff
My Tasks	/crm/my-tasks?staff=Name	Open tasks for this staff
Staff Dashboard	/crm/staff-dashboard	Per-staff summary cards
Staff Performance	/crm/staff-performance	Leaderboard
Performance Detail	/crm/staff-performance-detail/<name>	Deep-dive for one staff
Staff Workload	/crm/staff-workload	Workload distribution

---

11. TASK MANAGEMENT ARCHITECTURE

11.1 How Tasks Work

Tasks are **not stored in a separate DB table**. They are stored as `FOLLOW_UP_TASK` events in `lead_event` table with a JSON payload containing a unique `task_id`.

Create Task:

1. Admin fills task form in `/crm/lead/<phone>`
2. Backend generates `task_id = str(uuid4())`
3. Logs `FOLLOW_UP_TASK` event with JSON payload
4. Lead-level task_summary is derived by reading events

Complete Task:

1. Admin clicks Complete button
2. Backend logs `FOLLOW_UP_COMPLETED` event with `{"task_id": "..."}`
3. Template shows completed status

11.2 Task JSON Schema

```json
{
  "task_id": "550e8400-e29b-41d4-a716-446655440000",
  "task": "Call about PGDCA batch dates",
  "due_date": "2026-06-10",
  "staff": "Kiran",
  "phone": "919447329972"
}
```

11.3 Task Completion Detection

```python
# In crm_lead_detail():
task_map = {}
for ev in events:
    if ev.event_type == "FOLLOW_UP_TASK":
        task_map[task_id] = payload
    elif ev.event_type == "FOLLOW_UP_COMPLETED":
        task_map[task_id]["_completed"] = True

# Task is overdue if:
due_dt = datetime.strptime(due, "%Y-%m-%d")
overdue = (today.date() - due_dt.date()).days > 0
```

11.4 Task Routes (within admin.py)

Tasks are created/completed via POST endpoints within the lead detail route. The `/crm/my-tasks` and `/crm/admin-tasks` routes read from `lead_event` with `event_type IN ("FOLLOW_UP_TASK", "FOLLOW_UP_COMPLETED")`.

---

12. OPERATIONS INTELLIGENCE ARCHITECTURE

12.1 Phase 9.5 Operations Command Center (`/crm/operations`)

The operations page combines three intelligence layers:

1. **`calculate_operations()`** — Admission readiness, data issues, high-value opportunities, staff workload
2. **`calculate_intelligence()`** — 5-module deep intelligence (task intel, admission velocity, etc.)
3. **`calculate_automation_intelligence()`** — Task vs automation metrics

12.2 Admission Ready Logic

A lead is "admission ready" when:

```python
staff assigned AND
lead_score >= 60 AND
at least 1 course enquiry AND
NOT yet admitted
```

12.3 Operations Action Buckets

Bucket	Criteria
Admission Ready	staff assigned + score ≥ 60 + ≥1 enquiry + not admitted
High Value Ops	score ≥ 80 + ≥2 course enquiries + not admitted
Data Issues	admitted with no COURSE_ADMISSION event, unassigned leads, multiple enquiries+no admissions
Staff Workload	Per-staff: assigned count, hot count, admission-ready count, follow-up-needed count

12.4 Action Center Logic (`/crm/action-center`)

Bucket Priority (each lead goes to FIRST matching bucket):

1. **Admission Ready:** has_demo AND has_fees AND not admitted AND assigned
2. **HOT Leads:** score ≥ 80 AND not admitted
3. **Multi-Course Opportunities:** ≥ 3 course enquiries
4. **Demo Pending:** has_demo AND not admitted
5. **Follow-up Required:** assigned + not admitted + last activity > 3 days ago

---

13. AUTOMATION ENGINE RULES

13.1 Follow-Up Scheduler

The follow-up scheduler runs as a **daemon thread** polling the DB every **5 minutes**.

Templates:

Day	Hours	Message Theme
Day 1	24h	Soft check-in "Aaliza from Oxford"
Day 3	72h	Urgency — batch starting soon
Day 7	168h	Final follow-up + EMI offer

Skip Rule:

```python
# Skip if lead was active in the last 6 hours
if (now - last_active_dt).total_seconds() < 21_600:
    job.done = True  # skip, mark done
    continue
```

On Send:

1. `send_text(phone, message)` → WhatsApp API
2. `update_lead_status(phone, "Follow-up Day N Sent")` → thread
3. `log_message(...)` → MessageLog (outbound/followup)
4. `job.done = True` → DB commit

13.2 Campaign Engine

Campaign audience types:

```python
audiences = {
    "HOT Leads":           phones with temperature == "HOT",
    "WARM Leads":          phones with temperature == "WARM",
    "Demo Requested":      phones with DEMO_REQUESTED event,
    "Fees Requested":      phones with FEES_REQUESTED event,
    "Placement Interested": phones with PLACEMENT_ASKED event,
    "Needs Reply":         phones where last message was incoming,
    "Critical Leads":      phones with aging_status == "Critical",
    "All Leads":           all phones,
}
```

Campaign limits:
- Maximum recipients: **100** (hard limit, prevents accidental mass spam)
- Rate: ~1.5 seconds per recipient (estimated)

Campaign logging:
- Each message logged to `conversation_message` with `source="campaign"`
- Message body includes `[CAMPAIGN: {name}]` tag for tracking

13.3 Bot Trigger Keywords

Keywords	Action
`demo`, `free demo`, `free class`, `book demo`	→ demo flow, DEMO_REQUESTED event
`fees`, `fee`, `price`, `cost`, `ethra`, `how much`	→ fee display, FEES_REQUESTED event
`placement`, `job assistance`, `job guarantee`	→ placement info, PLACEMENT_ASKED event
`visit`, `office`, `varam`, `neritt`, `address`, `location`	→ office info, update status
`call me`, `call`, `counselor`, `phone`	→ call info, update status

| `hi`, `hello`, `hai`, `namaskaram` | → greeting → goal selection |

| `exit` | → goodbye message, stage=done |

| `courses`, `list`, `all courses` | → goal selection |

| `offer`, `discount` | → offer menu |

| `pay`, `payment`, `enrol`, `seat` | → payment flow |

13.4 Objection Handling

Objections are detected by `detect_objection(low)` and handled by `handle_objection(type, name, st)`:

Common objections handled:

- Price too high → EMI option, value pitch

- Not now / later → gentle follow-up suggestion

- Already studying → congratulate + suggest parallel course

- Location far → online class option

---

14. ADMISSION ANALYTICS FORMULAS

Route: `GET /crm/admission-analytics`

Function: `calculate_admission_analytics()`

14.1 Overall Conversion Rate

```python
overall_conversion = (total_admissions / total_leads * 100)
# Rounded to 1 decimal place
```

14.2 Staff Attribution Rules

- **Lead ownership** (Pipeline metric): current `assigned_staff` field
- **Admission credit** (Performance metric): staff from `COURSE_ADMISSION` event JSON `"staff"` field
- If no `COURSE_ADMISSION` event → falls back to `assigned_staff`

```python
# For leads pipeline count:
staff_stats[current_staff]["leads"] += 1

# For admissions credit:
admission_staff = admission_staff_map.get(phone, current_staff)  # event-first
staff_stats[admission_staff]["admissions"] += 1
```

14.3 Course Resolution Priority

```python
course_key = (course or "").strip() or (offer_course or "").strip() or "Unknown"
course_key = normalize_course_name(course_key)
```

14.4 Per-Staff Conversion

```python
conversion = (admissions / leads * 100)  # rounded to 1dp
```

14.5 Top Staff / Top Course

Only considers rows where `admissions > 0` (ignores zero-conversion staff/courses from headline KPI).

---

15. REVENUE ANALYTICS FORMULAS

Route: `GET /crm/revenue-analytics`

Function: `calculate_revenue_analytics()`

> [!WARNING]

> Revenue amounts are NOT stored in the database as of Phase 8.1. The `revenue_configured = False` flag triggers a warning banner on the dashboard. No fabricated revenue figures are shown.

15.1 Admission Percentage

```python
admitted_pct = (total_admissions / total_leads * 100)
# Rounded to 1 decimal place
```

15.2 Course Enquiry Aggregation

Events used: `COURSE_VIEWED`, `COURSE_ENQUIRY`, `COURSE_ADMISSION`

```python
# Per course:
enquiries  = len(unique phones with COURSE_VIEWED or COURSE_ENQUIRY)
# Note: COURSE_ADMISSION implies enquiry (counted in both)
admissions = len(unique phones with COURSE_ADMISSION)
conversion = (admissions / enquiries * 100)
```

15.3 When Revenue Will Work

When a payment tracking field is added (future phase), `revenue_configured = True` and `revenue_amount` can be summed per admission. The formula will be:

```python
total_revenue = sum(COURSE_FEES[course_name][0] for admitted leads)
```

---

16. RAILWAY DEPLOYMENT SETUP

16.1 Procfile

```
web: gunicorn run:app
```

16.2 run.py

```python
from app import create_app
app = create_app()

if __name__ == "__main__":
    app.run(debug=True)
```

16.3 Required Environment Variables on Railway

```
DATABASE_URL      = postgresql://... (auto-injected by Railway PostgreSQL plugin)
ACCESS_TOKEN      = WhatsApp Cloud API token (from Meta Developer Console)
PHONE_NUMBER_ID   = WhatsApp Phone Number ID
VERIFY_TOKEN      = oxford2026 (or custom)
ADMIN_KEY         = oxford_admin_2026 (or custom — CHANGE IN PRODUCTION)
GEMINI_API_KEY    = Google Gemini API key
SECRET_KEY        = Random 32-char string (for Flask sessions)
GOOGLE_CREDENTIALS = JSON string of service account credentials
SHEETS_ID         = Google Sheets document ID
```

16.4 First Deploy Steps

```bash
# 1. Push code to GitHub
git push origin main

# 2. Connect Railway to GitHub repo
# 3. Add PostgreSQL plugin in Railway dashboard
# 4. Set all environment variables
# 5. Deploy

# 6. Run database migrations (Railway console or via CLI):
flask db upgrade

# 7. Verify webhook
# WhatsApp Dashboard → Webhooks → Add callback URL:
# https://your-app.railway.app/webhook
# Verify token: oxford2026
```

16.5 Database Migrations

```bash
# Local:
flask db init      # Only first time
flask db migrate -m "Description"
flask db upgrade

# Railway (via CLI or Railway shell):
flask db upgrade
```

16.6 Health Check

Route: `GET /health`

Returns: `{"status": "ok", "version": "..."}` or similar uptime response.

Railway uses this route for deployment health verification.

16.7 Connection Pool Settings

```python
SQLALCHEMY_ENGINE_OPTIONS = {
    "pool_pre_ping": True,    # Detect stale connections before use
    "pool_recycle":  1800,    # Recycle connections every 30 minutes
}
```

---

17. PRODUCTION INCIDENT HISTORY & VERIFIED FIXES

17.1 Incident: "Working Outside Application Context" Error

Symptoms: Follow-up scheduler or webhook daemon threads crash with Flask context error.

Root Cause: Daemon threads trying to access `db` without a Flask app context.

Fix Applied:

```python
# All thread functions use _in_thread wrappers:
def log_message_in_thread(app, **kwargs):
    with app.app_context():
        log_message(**kwargs)

# Capture app ref in request context BEFORE spawning thread:
_app = current_app._get_current_object()
threading.Thread(target=log_message_in_thread, kwargs=dict(app=_app, ...)).start()
```

17.2 Incident: `postgres://` Connection String Error

Symptoms: SQLAlchemy fails to connect on Railway with `postgres://` URL.

Root Cause: Railway uses `postgres://` but SQLAlchemy requires `postgresql://`.

Fix Applied (in config.py):

```python
if DATABASE_URL.startswith("postgres://"):
    DATABASE_URL = DATABASE_URL.replace("postgres://", "postgresql://", 1)
```

17.3 Incident: Stale DB Connection (504 Timeout)

Symptoms: After Railway PostgreSQL restarts, app gets 500 errors until restarted.

Fix Applied: `pool_pre_ping: True` in SQLAlchemy engine options. This detects stale connections and reconnects automatically.

17.4 Incident: Course Name Mismatch in Analytics

Symptoms: Analytics showed "aidm" and "AIDM Digital Marketing" as separate courses.

Root Cause: Different code paths writing different aliases to `event_data`.

Fix Applied (Phase 7F.2):

```python
COURSE_NAME_ALIASES = {...}  # Normalized at READ time
def normalize_course_name(raw): ...
# Applied in all analytics functions before aggregation
```

17.5 Incident: Duplicate Staff Names Breaking Analytics

Symptoms: Staff "kiran" and "Kiran" appeared as two separate entries.

Root Cause: Case-sensitive string comparison on `assigned_staff`.

Fix Applied:

```python
def normalize_staff_name(name):
    return name.strip().title() if name and name.strip() else "Unassigned"
```

CRM Health dashboard also detects and flags duplicate naming variants.

17.6 Incident: Admission Without Staff Allowed

Symptoms: Leads marked as admitted with no assigned staff — analytics attribution impossible.

Fix Applied (Phase 8.2):

```python
```

```
# Hard block in crm_lead_update():
if new_admitted and not (lead.assigned_staff or "").strip():
    db.session.rollback()
    return redirect(url_for(..., err="Admission blocked: assign staff first"))
```

17.7 Incident: ConversationMessage Missing on Follow-up Sends

Symptoms: Follow-up messages showed in `message_log` but not in CRM timeline.

Root Cause: Follow-up scheduler only called `log_message()` (raw log), not `save_conversation_message()`.

Fix Applied: Added `log_message()` call in `_followup_worker()` after each send.

---

18. ROLLBACK PLAYBOOKS

18.1 Rollback New Feature (No Schema Change)

```bash
# 1. Identify the commit hash before the feature
git log --oneline -20

# 2. Create a rollback commit
git revert HEAD

# 3. Push to Railway
git push origin main
# Railway auto-deploys
```

18.2 Rollback New Feature (WITH Schema Change / Migration)

```bash
# STEP 1: Revert code to pre-migration state
git revert HEAD

# STEP 2: Downgrade database schema
flask db downgrade -1   # Go back one migration step

# STEP 3: Verify schema
# Check Railway PostgreSQL console

# STEP 4: Push code
git push origin main
```

> [!CAUTION]
> Never delete migrations files from the `migrations/` folder in production. Always use `flask db downgrade`.

18.3 Rollback Route-Only Change

For routes that add new HTML pages without DB changes:

1. Remove the route function from `admin.py`
2. Remove the nav link from templates
3. Delete the template file
4. `git commit -m "rollback: remove phase X route"`
5. `git push origin main`

18.4 Emergency: Restore from Backup

Railway PostgreSQL has automatic backups. Use Railway dashboard:

- Go to PostgreSQL plugin → Backups → Restore

18.5 Rollback Specific Files

```bash
# Revert a single file to previous version:
git checkout HEAD~1 -- app/routes/admin.py
git commit -m "rollback: revert admin.py to previous version"
git push origin main
```

---

19. ANTIGRAVITY CODING STANDARDS

These are the standards established throughout the Oxford CRM development that must be maintained in all future sessions.

19.1 Database Rules

1. **Never add per-row queries inside loops.** Use bulk queries + Python aggregation.
2. **Always guard new model queries** with try/except (migration may not be applied yet).
3. **LeadEvent is append-only.** Never UPDATE or DELETE lead_event rows.
4. **No foreign key constraints.** Use `phone` as soft join key.
5. **Use `db.session.rollback()` on exception** before returning error response.

19.2 Thread Safety Rules

1. **Always capture app ref before spawning thread:**

   ```python
   ```

```
_app = current_app._get_current_object()
threading.Thread(target=fn_in_thread, kwargs=dict(app=_app, ...)).start()
```

2. **All `_in_thread` functions open their own app context:**

```python
def fn_in_thread(app, **kwargs):
    with app.app_context():
        fn(**kwargs)
```

3. **Never access `db` from daemon threads without `app.app_context()`.**

19.3 Route Pattern Rules

1. **Always check `?key=ADMIN_KEY` first** and return `_deny()` (403) if missing.
2. **Use `redirect(url_for(...))` after POST**, never return HTML directly.
3. **Pass `msg=` and `err=` as query params** for flash-style messages.
4. **All analytics routes are read-only** — no writes in GET handlers.

19.4 Analytics Function Rules

1. **Maximum 2 bulk queries per analytics function.**
2. **All aggregation in Python memory**, not SQL GROUP BY.
3. **Function name format:** `calculate_<feature>()`
4. **Always return a plain dict** safe for Jinja2 template.
5. **Handle division-by-zero:** `round(x/y * 100, 1) if y > 0 else 0.0`

19.5 Course Name Rules

1. **Always call `normalize_course_name(raw)`** before comparing or aggregating course names.
2. **`COURSE_NAME_ALIASES` is read-only** — add aliases, never modify canonical names.
3. **Canonical names are defined in `ALL_COURSES`** (constants.py) — they are the source of truth.

19.6 Staff Name Rules

1. **Always call `normalize_staff_name(raw)`** before comparing staff names.

2. **Staff "Unassigned"** is the sentinel for unassigned leads.

3. **Never hardcode staff names** in logic — always read from `staff_master.json`.

19.7 Event Logging Rules

1. **Fire events from threads** (daemon threads), not in the main request path for performance.

2. **Use `log_lead_event_in_thread()` from request handlers**, `log_lead_event()` from within-context code.

3. **COURSE_ENQUIRY fires once per unique course** (check existing events before firing).

4. **COURSE_ADMISSION fires once per unique course** (idempotency check required).

19.8 Template Variables

Pass all required variables explicitly to `render_template()`. Never rely on global context. Always pass `key=request.args.get("key", "")` so nav links work.

---

20. FUTURE ROADMAP

20.1 Phase 9.7 (Planned)

- **Revenue amount tracking:** Add `fee_paid` / `payment_amount` field to `conversation_state` or a new `payment` table

- **WhatsApp template messages:** Pre-approved templates for batch notifications

- **Lead import:** CSV bulk import for offline leads

- **Advanced campaign scheduling:** Schedule campaigns for future datetime

20.2 Phase 10.0 (Planned)

- **Multi-branch support:** `branch_id` field in all tables for multiple Oxford locations
- **Google Calendar integration:** Auto-create calendar events for demo bookings
- **Lead scoring ML model:** Replace rule-based scoring with trained model
- **WhatsApp Business API v21.0 upgrade:** For new message types and reactions
- **Payment webhook:** Razorpay webhook to auto-confirm payment + set is_admitted=True

20.3 Phase 11.0 (Vision)

- **Student portal:** Post-admission student tracking (attendance, grades)
- **Certificate generation:** Auto-PDF certificate on course completion
- **Alumni network:** WhatsApp group management for batches
- **Predictive analytics:** Forecast next month's admissions from current pipeline
- **Mobile admin app:** React Native CRM app for staff on-the-go

---

21. TROUBLESHOOTING GUIDE

21.1 Bot Not Responding

Checklist:

```
1. Is Railway app running? → Check /health endpoint
2. Is webhook verified? → GET /webhook returns challenge
3. Is ACCESS_TOKEN valid? → Check WhatsApp API logs
4. Is PHONE_NUMBER_ID correct? → Check config.py
5. Check Railway logs for "❌ Webhook error:"
```

21.2 Database Connection Error

```bash
# Verify DATABASE_URL is set:
# Railway → Variables tab → DATABASE_URL

# Check it starts with postgresql:// not postgres://
# config.py handles this automatically

# Test connection:
flask shell
>>> from app.extensions import db
>>> db.engine.execute("SELECT 1")
```

21.3 CRM Page Shows "Access Denied"

Cause: Missing or wrong `?key=` parameter.
Fix: Append `?key=oxford_admin_2026` (or your ADMIN_KEY) to the URL.

21.4 Analytics Showing Wrong Counts

Check 1: Are course names normalized?
```python
# In Flask shell:
from app.bot.constants import normalize_course_name
normalize_course_name("aidm")  # Should return "AIDM Digital Marketing"
```

Check 2: Are staff names normalized?
```python
```

```
from app.routes.admin import normalize_staff_name
normalize_staff_name("kiran")  # Should return "Kiran"
```

Check 3: Run CRM Health check → `/crm/health` — shows all data quality issues.

21.5 Follow-up Scheduler Not Running

```python
# Check in Flask shell or logs:
from app.services.followup_service import scheduler_started
print(scheduler_started)  # Should be True after first poll (5 min)

# Check Railway logs for:
# "✅ Follow-up scheduler started (DB-backed)"
# "📤 Follow-up Day 1 → StudentName"
```

21.6 Migration Error on Deploy

```bash
# Common error: relation "X" already exists
# Fix: Check current migration head
flask db current

# If DB is ahead of code:
flask db stamp head  # Mark DB as current without running migrations

# If DB is behind:
flask db upgrade
```

21.7 WhatsApp Message Not Sending

```python
# Check whatsapp_service.py:
r = send_text(phone, message)
print(r.status_code, r.text)

# Common errors:
# 400: Invalid phone format (must start with country code, no +)
# 401: Invalid ACCESS_TOKEN
# 131030: Message template not approved
```

21.8 Gemini AI Not Responding

```python
# Check GEMINI_API_KEY is set
# Check ai_service.py for fallback:
# If gemini_reply() returns None → smart_fallback() is used
# smart_fallback() always returns a valid string
```

21.9 Staff Deactivation Blocked

Error: `"BLOCK_DEACTIVATION:{count}:{name}"`
Meaning: Staff has leads assigned; cannot deactivate.
Fix: Reassign all leads to another staff first (use Reassignment Center), then deactivate.

---

22. CHANGE LOG BY PHASE

Phase	Description	Key Changes
Phase 1	Basic WhatsApp bot	Webhook, smart_reply(), in-memory state dict
Phase 2	Course catalog	ALL_COURSES, GOAL_COURSES, OFFER_MENU, payment links
Phase 3	DB persistence	ConversationState, FollowUpJob, StateProxy, scheduler
Phase 4A	CRM fields	lead_status, assigned_staff, lead_score, is_admitted, notes
Phase 4B	Admin panel	/panel route, basic lead list
Phase 4C	Lead detail	/crm/lead/<phone> route
Phase 4D	Message log	MessageLog table, log_message()
Phase 5A	CRM timeline	ConversationMessage table, save_conversation_message()
Phase 5B	CRM updates	crm_lead_update(), crm_lead_send()
Phase 5C	Search & filters	Conversation search, source filter, date range
Phase 6A	Lead events	LeadEvent table, COURSE_VIEWED, FEES_REQUESTED, DEMO_REQUESTED, PLACEMENT_ASKED
Phase 6B	Intelligence engine	calculate_lead_intelligence(), EVENT_SCORE_MAP
Phase 6C	Intelligence caching	Batch intelligence on /crm/leads page
Phase 6D	Lead health	calculate_lead_health(), aging_status, escalation
Phase 6E	Unified timeline	Merge events+messages sorted by created_at
Phase 6F	Health indicators	needs_reply, has_pending_fu badges
Phase 6G	Campaigns	/crm/campaigns, audience calculation, campaign send
Phase 7A	Funnel analytics	/crm/analytics, calculate_funnel_metrics()
Phase 7B	Staff performance	/crm/staff-performance, leaderboard
Phase 7C	Source analytics	/crm/source-analytics, attribution logic
Phase 7D	Admission analytics	/crm/admission-analytics, staff/course breakdown
Phase 7E	Course journey	get_course_enquiries(), get_course_admissions()
Phase 7F.2	Course normalization	COURSE_NAME_ALIASES, normalize_course_name()
Phase 8.1	Revenue analytics	/crm/revenue-analytics (revenue_configured=False)
Phase 8.2	Admission rules	Staff hard block, auto-promote to Enrolled
Phase 8.3A	Multi-course admission	/crm/course-admissions/<phone>

| **Phase 8.4** | Lead portfolio | calculate_lead_portfolio(), per-lead summary |

| **Phase 8.5** | CRM health | /crm/health, data quality checks |

| **Phase 8.6** | Action center | /crm/action-center, 5 priority buckets |

| **Phase 8.8** | Operations center | /crm/operations, staff workload |

| **Phase 9.1** | Audit events | LEAD_REASSIGNED, MANUAL_MESSAGE, ADMISSION_OWNER |

| **Phase 9.2A** | Staff management | /crm/staff-management, staff_master.json |

| **Phase 9.3A** | Task management | FOLLOW_UP_TASK, FOLLOW_UP_COMPLETED events |

| **Phase 9.5** | Operations intel | calculate_intelligence() — 5 modules |

| **Phase 9.6** | Automation intel | calculate_automation_intelligence() |

APPENDIX A: Quick Reference Card

All CRM URL Patterns

```
/crm/leads?key=KEY

/crm/lead/919447329972?key=KEY

/crm/analytics?key=KEY

/crm/staff-performance?key=KEY

/crm/source-analytics?key=KEY

/crm/admission-analytics?key=KEY

/crm/revenue-analytics?key=KEY

/crm/health?key=KEY

/crm/action-center?key=KEY

/crm/operations?key=KEY

/crm/staff-management?key=KEY

/crm/campaigns?key=KEY

/crm/my-leads?key=KEY&staff=Kiran

/crm/my-tasks?key=KEY&staff=Kiran

/crm/admin-tasks?key=KEY
```

/crm/unassigned-leads?key=KEY

/crm/reassignment-center?key=KEY

/crm/staff-dashboard?key=KEY

/crm/staff-workload?key=KEY

/panel?key=KEY

/stats (X-Admin-Key header)

/health

```

### Event Type Quick Reference

```

LEAD_CREATED   → +2 pts (new phone, auto)

FIRST_MESSAGE_RECEIVED → +3 pts (new phone, auto)

AI_RESPONSE_SENT   → +5 pts (new phone, auto)

COURSE_VIEWED   → +10 pts (bot, auto)

PLACEMENT_ASKED   → +15 pts (bot, auto)

FEES_REQUESTED   → +20 pts (bot, auto)

DEMO_REQUESTED   → +25 pts (bot, auto)

PAYMENT_PENDING   → +30 pts (bot, auto)

COURSE_ENQUIRY   → 0 pts   (admin update)

COURSE_ADMISSION   → 0 pts   (admin mark admitted)

LEAD_REASSIGNED   → 0 pts   (admin reassign)

MANUAL_MESSAGE   → 0 pts   (admin send message)

FOLLOW_UP_TASK   → 0 pts   (admin create task)

FOLLOW_UP_COMPLETED   → 0 pts   (admin complete task)

```

### Temperature Thresholds

```

score ≥ 80 → 🔴 HOT

score ≥ 50 → 🟡 WARM

score < 50 → 🔵 COLD

```

### Aging Status Thresholds

```
0–2 days → 🟢 Fresh

3–6 days → 🟡 Attention

7–13 days → 🟠 Aging

14+ days → 🔴 Critical

```

---

## APPENDIX B: Contact & Infrastructure

| Item | Value |
|---|---|
| Institute | The Oxford Computers |
| Location | Malayinkeezhu Junction, Thiruvananthapuram, Kerala |
| Phone | 9447329972 |
| Website | theoxfordedu.com |
| WhatsApp Bot Number | (configured via PHONE_NUMBER_ID) |
| Hosting | Railway (railway.app) |
| Database | Railway PostgreSQL plugin |
| AI | Google Gemini API |
| Payments | Razorpay (razorpay.com) |
| Certification | Kerala State Rutronix Approved |

---

*End of Oxford CRM Master Technical Handbook — Version Phase 9.6*
```

Generated by Antigravity AI | Last updated: June 2026